# Appendix B

# How to Add a Code to AMUSE

> Jag har aldrig provat det förut, så jag tror absolut att jag borde kunna göra det.
>
> I have never tried that before, so I think I should definitely be able to do that.
>
> —Pippi Långstrump (apocryphal)

The main part of this manuscript presents an extended description of how to use the AMUSE framework. Here, we focus on the basic structure and design of AMUSE.

## B.1 The AMUSE Framework

At first sight, the approach of assimilating existing software into a larger framework would appear to be a difficult undertaking, particularly since community software is written in a wide variety of languages, such as FORTRAN, C, and C++, and exhibits an enormous diversity of internal units and data structures. On the other hand, such a framework, if properly designed, could be relatively easy to use, since learning one simulation package is much easier than mastering the idiosyncrasies of many separate programs. The use of standard calling procedures can enable even novice researchers to quickly become acquainted with the environment and to perform complicated simulations using it.

Within AMUSE, a user writes a relatively simple script with a standardized calling sequence to each of a set of underlying *community codes*. Instructions are communicated through a message-passing interface to any number of spawned community modules, which respond by performing operations and transferring data back through the interface to the user script. This appendix is intended to help you write production-quality (meaning working and clearly written) user scripts. There

doi:10.1088/2514-3433/adfc3cch13

are numerous examples in the amuse/examples directory, and here we discuss several of them (see for example the code examples in Chapters 4 and 5).

As illustrated in Figure B.1, the two top-level components of AMUSE are:

- The *user script*, which implements a specific physical problem or set of problems, in the form of system-provided or user-written scripts that serve as the user interface to the AMUSE framework. Coupling between community codes is implemented at this level only, with the help of support classes provided by the *manager*.
- The *community module*, comprising three key elements:
    1. The *manager*, which provides an object-oriented interface onto the *communication* layer via a suite of system-provided utility functions.

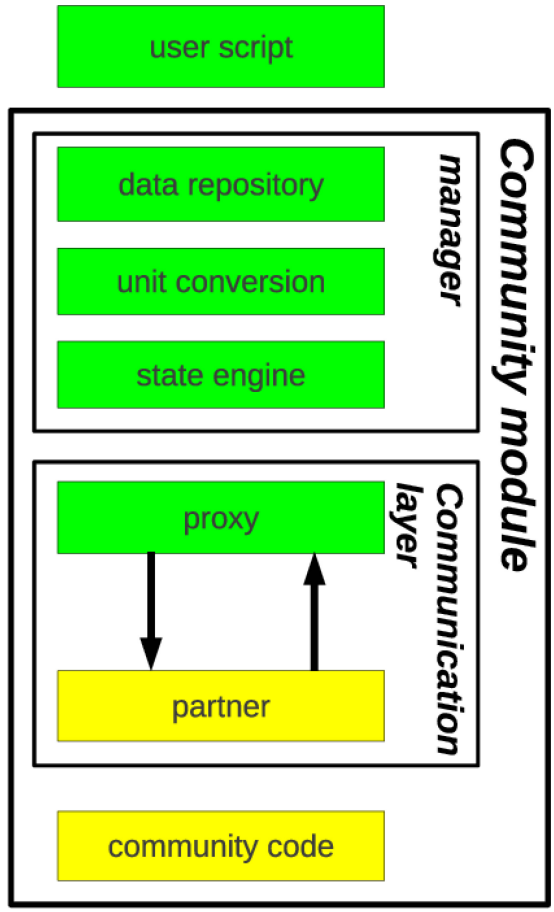


**Figure B.1.** The AMUSE framework architecture is based on a three-layered design. The top layer consists of a user script written by the end user in Python (top box). The *manager* is part of the *community module* and consists of a data repository, unit conversion, and a state engine. The *community module* also includes the partner–proxy pair in the communication layer—a bidirectional message-passing framework—and a community code (bottom box).

This layer handles unit conversion, as discussed below, and also contains the state engine and the associated data repository, both of which are needed to guarantee consistency of data across modules. The role of this layer is generic—it is not specific to any particular problem or to any single physical domain. In the rest of this manuscript, we discuss an actual implementation of the *manager* for astrophysical problems and give some examples of coupling codes.

2. The *communication* layer, which realizes the bidirectional communication between the *manager* and the community code layer. This is implemented via a proxy and an associated partner, which together provide the connection to the community code.
3. The *community code* layer, which contains the actual community codes and implements control and data management operations on them. Each piece of code in this layer is domain-specific, although the code may be designed to be very general within its particular physical domain.

### B.1.1 The Community Module

The community module consists of the actual community code and the bidirectional communication layer enabling low-level MPI (Gropp et al. 1996) communication between the proxy and the partner. Each community module contains a Python class of functions dedicated to communication with the community code. Direct use of this low-level class from within a user script is possible but discouraged because it requires considerable replication of code dealing with data models, unit conversion, and exception handling. Instead, the AMUSE framework provides the manager layer on top of the community module layer to manage the bookkeeping needed to maintain the interface with the low-level class.

#### *B.1.1.1 The Community Code*

Community applications may be programmed in any computer language that supports smart sockets or bindings to the MPI protocol. In practice, these codes exhibit a wide variety of structural properties, model diverse physical domains, and span a broad range of temporal and spatial scales. In AMUSE this diversity is exemplified by a suite of community programs ranging from toy codes to production-quality, high-precision dedicated solvers. A user script can be developed quickly using toy codes until the production and data analysis pipelines are fully developed, and can then easily be switched to production-quality implementations of the physical solvers for actual large-scale simulations.

In principle, each of the community modules can be used standalone, but the main strength of AMUSE is in the coupling between them. Many of the codes incorporated in our practical implementation were not written by us, but are publicly available and are amply described in the literature.

*B.1.1.2 The Communication Layer*

Bidirectional communication between a running community process and the manager could in principle be realized using any protocol for remote interprocess communication. Our main reasons for choosing MPI are similar to those for adopting Python—widespread acceptance in the computational science community and broad support by many programming languages and on most computing platforms.

The communication layer consists of two main parts, a Python proxy on the manager side of the framework and a partner on the community code side. The proxy converts Python commands into MPI messages, transmits them to the partner, and then waits for a response. The partner waits for and decodes MPI messages from the proxy into community code commands, executes them, and then sends a reply. The MPI interface allows the user script and community modules to operate asynchronously and independently. As a practical matter, the detailed coding, decoding, and communications operations in the proxy and the partner are never hand-coded. Rather, they are automatically generated as part of the AMUSE build process, from a high-level Python description of the community module interface.

We note that other solutions for linking high-level languages within Python exist —e.g. `swig` (see http://www.swig.org) and `f2py` (Peterson 2009), and in fact both were used in earlier implementations of the AMUSE framework (Portegies Zwart et al. 2009). However, despite their generality, these solutions cannot maintain namespace independence between community modules, and in addition are incapable of accommodating high-performance parallel community codes. For these reasons, we abandoned the standard solutions in favor of our custom, high-performance MPI alternative.

The use of MPI may seem like overkill in the case of serial operation on a single computing node (and this might well be the case), but it imposes negligible overhead, and even here it offers significant practical advantages. It rigorously separates all community processes in memory, guaranteeing that multiple instances of a community module are independent, and explicitly avoids namespace conflicts. It also ensures that the framework remains thread-safe when using older community codes that may not have been written with that concept in mind.

Despite the initial threshold for its implementation and use, the relative simplicity of the message-passing framework allows us to easily couple multiple independent community codes as community modules, or multiple copies of the same community module, if desired (see Figure B.1). An example of the latter might be to simulate simultaneously two separate objects of the same type, or the same physics on different scales, with different boundary conditions, without having to modify the data structures in the community code. Specifically, we might want to evolve two stars of the same mass but with different metallicities.

As a bonus, the framework naturally accommodates inherently parallel community modules and allows simultaneous execution of independent modules from the user script. Individual community modules can be offloaded to other processors, in a cluster or on the grid, and can be run concurrently without requiring any

changes in the user script, which still looks like a simple serial code. This makes the AMUSE framework well suited for distributed computing with a wide diversity of hardware architectures.

*B.1.1.3 The Manager*

Between the user script and the communication layer, we introduce the manager, which as part of the community module. This part of the code is visible to the script-writing user and guarantees that all accessible data are up to date, have the proper format, and are expressed in the correct units.

The manager layer constructs data models (e.g., particle sets or tessellated grids) on top of all functions in the low-level communication class and checks the error codes they return. To allow different modules to work in their preferred systems of units, the manager also incorporates a unit-conversion protocol (see Section B.1.3). It also includes a state engine to ensure that codes are not in the middle of other activities when they hand control back to the framework, since many community codes are written with specific assumptions about calling procedures. The state engine is important, for example in adding particle to a tree code: for performance, it is much better to delay rebuilding the tree until all particles have been added rather than rebuilding the tree every time a particle is added. This repository can be structured in one or more native formats, such as particles, grids, tessellation, etc., depending on the topology of the data adopted in the community code.

Each community code has its own internal data, needed for modeling operations but not routinely exposed to the user script. However, some of these data may be needed by other parts of the framework. To share data effectively and to minimize the bookkeeping for data coherency, the most fundamental parts of the community data are replicated as a structured repository in the manager. The internal data structures differ, but the repository imposes a standard format. It allows the user to access community module data from the user script without having to make individual (and often idiosyncratic) calls to individual community modules. The repository is updated from the community module on demand and is considered to be authoritative within the script.

The separation of the manager from the community module realizes a flexibility that allows the user to swap modules and recompute the same problem using different physics solvers, providing an independent check of the implementation, validation of the model, and verification of the results, simply by rerunning the same realization of the initial conditions using another community module. Potentially even more interesting is the possibility of solving some parts of a problem with one solver and others with a different solver of the same type, and then combining the results at a later stage to complete the solution. While studying the general behavior of a simulation, perhaps to test a hypothesis or guide our physical intuition, we may elect to use computationally cheaper physics solvers, and then switch to more robust, but computationally more expensive, modules in a production run. Alternatively, we might adopt less accurate solvers for physical domains that are not deemed crucial for capturing the correct overall behavior.

A further advantage of separating the manager from the community module is the possibility of combining existing community modules into new community modules which can themselves be coupled via the manager, building in this way a hierarchical environment for simulating complex physical systems with multiple components.

### B.1.2 The User Script

The AMUSE framework is controlled by the user via Python scripts. The main tasks of these scripts are to identify and spawn community modules, control their calling sequences, and manage the data flow among them. The user script and the spawned community code are connected by a communication channel embedded in the community module and maintained by the manager. Since the community modules do not communicate directly with one another, the only communication occurs between modules and the user script. The number of community modules controlled by a user script is effectively unlimited, and multiple instances of identical modules can be spawned.

The main objective of the user script is to read in or generate an input model, process these initial conditions through one or more simulation modules, and subsequently mine and analyze the resultant data. In practice, the user script will itself be composed of a series of scripts, each performing one specific functionality.

### B.1.3 Intermodule Data Transfer and Unit Conversion

Computer scientists tend to attribute types to parameters and variables, and generally impose strict rules for the conversion from one type to another. In physics, data types are almost never important (everything is real or integer), but units are crucial for validating and checking the consistency of a calculation or simulation. The lack of coherent unit handling in programming environments is notorious, and can cause significant problems in multiphysics simulations, particularly for inexperienced researchers.[1]

In many cases, a researcher can (and arguably should) define a set of dimensionless variables applicable to a specific simulation, within the confines of a set of rather strict assumptions. For a monophysics simulation, the consistent use of such variables is generally clear to most users. However, as soon as an expert from another field attempts to interpret the results, or when the output of one simulation is used by another program, units must be restored and the data converted to another physical system for further processing and/or interpretation.

The absence of units in scientific software raises two important problems. The first is the loss of an independent consistency check for theoretical calculations. The second is the expert knowledge and intuition required to manage dimensionless variables or otherwise unfamiliar units. Few would recognize $4.303 \times 10^{43}$ as the

[1] The Mars Climate Orbiter crashed on 1998 December 11 on the red planet due to a miscalculation during Trajectory Correction Maneuver 4 in the conversion of imperial unit pound-seconds adopted by NASA to the metric units of Newton-seconds used by Lockheed.

radius of the Sun in units of the Planck length, even though it is not completely inconceivable that an astronomer might use such units in a cosmological simulation. To address these issues, AMUSE incorporates a unit-conversion module as part of the manager (see Section B.1.1.3) to guarantee that all communications within the top-level user script are performed in the proper units. In order to prevent unit checking and conversion from becoming a performance bottleneck, we adopt "lazy" evaluation, performing unit conversion only when explicitly required.

### B.1.4 Installing Prerequisites and Debugging

The non-Python packages on which AMUSE depends are listed in Table B.1. These dependencies may differ in later releases; so to be sure that you install the correct packages, it is best to consult the AMUSE web site https://www.amusecode.org. Many of these dependencies may already be installed in recent `Linux` and `macOS` releases, but some fine-tuning may be needed to complete the dependency installation.

The Message Passing Interface (or MPI for short) is an important package, although the same functionality can be realized using sockets (Gay 2000). We recommend using either sockets, `OpenMPI` (Dagum & Menon 1998) or `MPICH` (Gropp 2002). The former has the advantage that the communication does not require a process when running, which saves computer time for other tasks, but the latter is more standard, usually faster, and you will then have an MPI running for other parallel tasks.

When running on a supercomputer, it is probably best to use the native `numpy` (van der Walt et al. 2011) and the MPI interface specific to the architecture rather

**Table B.1.** Dependencies Needed to Compile and Run AMUSE

| Package Name | Minimum Version Number | Note |
|---|---|---|
| Python | 3.7 | Scripting programming language |
| `HDF5` | 1.6.5 | Little/big endian neutral binary formatted I/O |
| `MPI` | N/A | Message-passing interface |
| `FFTW` | 3.0 | Subroutine library for discrete Fourier transforms |
| `GSL` | 2.0 | Scientific Library for C and C++ programmers |
| `CMake` | 2.4 | Cross-platform, open-source build system |
| `GMP` | 4.2.1 | Library for arbitrary precision arithmetic |
| `MPFR` | 2.3.1 | C Library for multiple-precision floating-point computations |
| `Numpy` | 1.18.0 | Numerical toolbox for Python |
| `h5py` | 3.0 | HDF5 interface for Python |
| `mpi4py` | 3.0 | Python interface for MPI |
| `pytest` | 3.0 | Extension of the Python testing framework |
| `docutils` | 0.18 | Documentation support for Python packages |

than installing your own, but for all other packages this should not make a difference. When installing AMUSE on multiple machines in order to use distributed operation (using `ibis`–see Section B.1.6; Bal et al. 2010; Drost et al. 2012), it is important to ensure that the each of the machines runs the same version of `Java` (Gosling et al. 1996, 2014); otherwise, the distributed interface will become confused and your scripts will fail with unpleasant distributed error messages, which are hard to debug.

### B.1.5 Reproducibility with `git`

A nice feature of `git` is that older versions are guaranteed to exist, and can be downloaded and installed if needed. In this way, we can, for example, ensure that the version of AMUSE we are using is identical to the version used to write this book. To check the version number, you can use the command

```
>% git rev-parse HEAD
```

which, on the day we submitted the manuscript to the publisher, gave

```
8d81c91a9fbd55b7510b3949700d3a594e5cc179
```

or version v2025.5.0.rc1 from 2025 May 30. This is the unique key for the particular version of AMUSE under which we tested many of the scripts in the book. If you wish to use exactly the same version of `amuse`, you can download and install it using the bash script presented in Code example B.1. The script automatically downloads and starts the script that will guide you through installing the AMUSE package.

Note, by the way, that we test AMUSE frequently to ensure that our scripts and test suites run as expected, so there is no actual reason to use only this specific version. However, for purposes of code verification, it is often important to be able to specify precisely the version of the software that was used to obtain a specific result, so this feature of `git` is key to writing reproducible code.

```
#! /bin/bash

GITHASH=450d20c0b542be53ff6fdf82bb6c2cce1c80c7bf
temp=amuse-temp-$$
mkdir $temp; cd $temp

git clone https://github.com/amusecode/amuse.git
cd amuse; git reset --hard $GITHASH; cd ../..

export AMUSE_DIR="$PWD/amuse-$GITHASH"
mv $temp/amuse $AMUSE_DIR
rm -rf $temp

cd $AMUSE_DIR
./setup
```

**Code example B.1:** Automated script to download a specific version of AMUSE. The script is available via `wget https://amusecode.org/get_amuse_and_install.sh`.

### B.1.6 Tuning Your AMUSE Installation

The operation of AMUSE can be controlled and tuned using environment variables. These are generally stored in your home directory in the file `.amuserc`. These settings are superseded by the same named file in the directory from which you launch your AMUSE script.

The file `.amuserc` can contain a number of directives. These include the choice between `MPI`, `sockets`, or `ibis`. `MPI` is the default; to use sockets as your main communication channel with the following lines in your `.amuserc` file:

```
[channel]


channel_type=sockets
```

The `ibis` channel allows the user to run `amuse` in distributed mode using the `ibis` package.

Another option is to redirect the output of any of the AMUSE modules you may be running. The native output of the community codes is usually discarded, this would be reflected by

```
redirection=null
```

but by writing

```
redirection=none
```

it is redirected to `stdout`. An alternative would be to write `file` instead of `null` or `none`. This latter option may produce a number of files, depending on the codes you run. The two lines

```
redirect_stdout_file=code.out

redirect_stderr_file=code.out
```

will redirect the standard output and error channels `stdout` and `stderr`.

A debugger, in this example the GNU debugger `gdb` (see Robbins 2005), may be enabled by adding the line

```
debugger=gdb
```

to the `.amuserc` file. The standard setting is debugger=none, indicating that no debugger is to be used. Alternatives are `xterm` for debugging with an `xterm` screen, `ddd` for a `ddd` session using a graphical debugger, or valgrind (Nethercote & Seward 2007) to run the community codes under `valgrind` to check for memory leaks. We present more detail on debugging options in Section B.1.7.

These options can also be provided as part of the argument list for the various modules at runtime. For example, one could start a *N*-body code `ph4` with the following arguments

```
gravity = Ph4(redirection="file", number_of_workers=16, mode="gpu",
              debugger="gdb")
```

to redirect the code output to the standard files identified in the original source code, run the code concurrently on 16 cores, each equipped with a GPU, and run the debugger in order to track numerical problems.

Several more options can be specified in the `.amuserc` file, but these are less relevant for the day-to-day operation of the package. An example of an `.amuserc` file is provided in the installation directory in your AMUSE repository.

### B.1.7 Debugging in AMUSE

Debugging a monophysics solver is sometimes difficult, but debugging an AMUSE script when the problem arises in one of the coupled codes can be much more challenging. With any luck, you will never have to do this because we hope that the authors of the individual community codes will continue to maintain their packages, but you never know.

There are several ways to debug a community code. Most basic is to set the code's internal "verbosity" level to a high value. Almost all codes have something like this, but there is absolutely no consensus on what value that should be because every author has a private method to do this. Since the community codes in AMUSE are (almost) virgin, meaning that we have touched them as little as possible while integrating them into the framework, the private verbosity settings are unchanged. However, apparently due to some unwritten understanding, setting the verbosity of most codes to a large positive integer value leads to an immense flow of information from the running code. This is illustrated in the example below:

```
self.hydro = Fi(converter, redirection="none")
self.hydro.parameters.verbosity = 99
```

Combining high verbosity with the `redirection` argument (indicating that the code's standard output should be written to your terminal) and running the code again generally produces enough (probably too much) information to debug obvious bugs.

A more systematic way is to start an online debugger. although this generally requires significant knowledge of the community code. In the following snippet, we start the `GNU` debugger `gdb` by providing it as an argument in the code (see also Section B.1.6):

```
hydro = Fi(converter, debugger="gdb")
```

**Table B.2.** Debugging Directives

| Method | Explanation |
|---|---|
| None | No debugger, normal run |
| gdb | Start a gdb debugger in a separate xterm |
| xterm | Start the code under xterm, ideal for console logging |
| ddd | Start a ddd session (a graphical debugger) |
| valgrind | Check for memory leaks at runtime (Nethercote & Seward 2007) |
| gdb-remote | Run gdb as a server that listens on a port<br>For remote debugging specify a port to listen to |
| debugger | None |
| debugger | gdb-remote |
| debugger_port | 4343 |

You can also specify these arguments in the `.amuserc` file in your root directory, or the directory from which you run the AMUSE script (see Section B.1.6). There are several options, as listed in Table B.2.

## B.2 Other Associated Codes and Packages

Many people contributed to AMUSE. They helped with development, maintenance, expansion, bug fixes, and provided interesting examples. Without the community, AMUSE would only be a shadow of its current form. Here, we discuss a few of the more prominent packages, not as big as `PeTar`, `Torch`, `Ekster`, or `Venice`, but still important. Many of these packages are still in development—honestly, all software is continuously in development. But they have been used in scientific production.

### B.2.1 TRES: Triple-Star Evolution

The Triple-star Evolution Code (`Tres`) (Toonen et al. 2012, 2016, 2020) is a numerical simulation framework designed to model the evolution of hierarchical triple systems, including both stellar and planetary components. `Tres` combines stellar evolution (e.g., changes in mass, radius, and luminosity) using the `SeBa` package with secular three-body orbital dynamics (Hamers & Portegies Zwart 2016), allowing for mutual influences between the stars' evolution and their orbital interactions.

In `Tres`, three stars with orbits are integrated synchronously. The stellar evolution is realized with `SeBa`, as well as the mass-transfer recipes between two stars. The tertiary orbit, and its effect on the inner binary, is integrated using the secular dynamics code. In addition, `Tres` accounts for the tidal evolution of the three stars, which is important because of von Zeipel–Lidov–Kozai cycles (von Zeipel 1910; Lidov 1962; Kozai 1962) can render the triple highly eccentric.

`Tres` is particularly useful for population synthesis studies, enabling researchers to explore the evolutionary pathways and interaction rates in large samples of triple systems, as presented in Toonen et al. (2020).

```
from tres import run_tres, initialize_triple_class
from tres.setup import make_particle_sets


inner_primary_mass = 10.0 | units.MSun
inner_secondary_mass = 8.0 | units.MSun
outer_mass = 5 | units.MSun
inner_semimajor_axis = 1.0 | units.au
outer_semimajor_axis = 12.0 | units.au
inner_eccentricity = 0.5
outer_eccentricity = 0.5
relative_inclination = np.pi / 3
inner_argument_of_pericenter = 0.0
outer_argument_of_pericenter = 0.0
inner_longitude_of_ascending_node = 0.0

stars, bins, correct_params = make_particle_sets(
    inner_primary_mass,
    inner_secondary_mass,
    outer_mass,
    inner_semimajor_axis,
    outer_semimajor_axis,
    inner_eccentricity,
    outer_eccentricity,
    relative_inclination,
    inner_argument_of_pericenter,
    outer_argument_of_pericenter,
    inner_longitude_of_ascending_node,
)

stellar_code = Seba()
secular_code = Seculartriple()


triple = initialize_triple_class(
    stars, bins, correct_params, stellar_code, secular_code
)

triple.stellar_code.parameters.metallicity = 0.02
triple.evolve_model(10 | units.Myr)
triple.print_stellar_system()
```

**Code example B.2:** Minimal example to set up a simulation using `Tres` (see `amuse.examples.tres_minimal`).

Code example B.2 shows a small example script on how to set-up a simulation using `Tres`. This is considerably more elaborate, mainly because now we have three stars and two orbits to initialize.

### B.2.2 AGAMA: Action-based Galaxy Modelling Architecture

AGAMA is an open-source toolkit for generating realistic *N*-body representation of galaxies (Vasiliev 2019); see also Vasiliev (2018). It offers a suite of computational methods for galactic modeling and analysis. It was developed since 2015 and is documented in over 130 pages of technical specifications.

AGAMA is a comprehensive software library for stellar dynamics and galactic modeling. Its core functionality centers on the computation of gravitational potentials and forces, which can be derived from analytic density profiles or $N$-body simulations using spherical-harmonic expansions and adaptive solvers. AGAMA supports the numerical integration of stellar orbits in both static and time-dependent potentials, accommodating composite systems such as interacting galaxies.

The library provides robust tools for coordinate transformations, enabling conversion between position-velocity and action-angle variables via the Stäckel approximation or torus mapping techniques. AGAMA implements a range of action-based distribution functions for disk and spheroidal systems, facilitating predictions of velocity distributions and the construction of self-consistent, multi-component galaxy models.

Scientifically, AGAMA is suited for galactic structure reconstruction using Schwarzschild orbit-superposition methods, dark matter halo profiling from stellar kinematics, equilibrium initial condition generation for $N$-body simulations, and the analysis of tidal streams and dynamical populations. The framework also supports parameter optimization against observational data.

Technically, AGAMA is written in C++ with performance optimizations using OpenMP parallelization. It offers a native Python API for interactive analysis and is interoperable with other astrophysical software ecosystems such as AMUSE, GALPY, and NEMO. The package is modular, allowing users to extend its capabilities with custom potential models and distribution functions. Included are advanced mathematical utilities for spline interpolation and multidimensional sampling, as well as extensive test suites and example workflows.

### B.2.3 Vader: Viscous Accretion Disk Evolution Resource

We implemented an interface to the VADER viscous evolution code (Krumholz & Forbes 2015) within the AMUSE framework. Our interface supports EPE using mass loss rates from the FRIED grid (Section B.2.4), and we extended the code to include IPE with mass loss profiles from Picogna et al. (2019), scaled by stellar mass using the relations in Owen et al. (2012). Stellar X-ray luminosities, which drive IPE rates, are based on Flaccomio et al. (2012) and decrease over time, as given by Equation B.4.

To model accretion onto the central star, we apply a time-dependent accretion rate following viscous disc theory (Lynden-Bell & Pringle 1974):

$$\dot{\mathrm{M}}_{\mathrm{accr}}(t) = \frac{\mathrm{M_d}}{2t_s}\left(1 + \frac{t}{t_s}\right)^{-3/2}, \tag{B.1}$$

with viscous timescale

$$t_s = \frac{R_d^2}{3}\frac{\Omega}{\alpha c_s^2}, \tag{B.2}$$

which we approximate using the model's temperature profile and ideal gas law:

$$t_s = 11\ \mathrm{Myr}\left(\frac{\alpha}{10^{-3}}\right)^{-1}\left(\frac{R_d}{200\ \mathrm{au}}\right)\left(\frac{M_*}{M_\odot}\right)^{1/4}. \tag{B.3}$$

Rather than explicitly evaluating Equation B.1, we scale the input accretion rate by $(1 + t/t_s)^{-3/2}$.

Boundary conditions are handled dynamically: when the inner mass flux is insufficient to sustain the prescribed accretion, we first attempt to reduce the accretion rate to empty the innermost cell; if this fails, we switch to a vanishing-torque condition.

EPE rates are fixed per model and scaled linearly with disc radius, with optional interpolation from the FRIED grid (Haworth et al. 2018) under a fixed 10 $G_0$ radiation field. Dust is evolved as a passive scalar, initially 1% of the gas mass, and subject to EPE following Haworth et al. (2018) with an exponential decay representing grain growth.

The implementation uses a logarithmic grid of 330 radial zones from 0.01 to 3000 au and assumes a turbulent viscosity parameter of $\alpha = 10^{-3}$.

### B.2.4 FRIED: Far-ultraviolet Radiation Induced Evaporation of Discs

EPE occurs when nearby stellar radiation heats disc material until it becomes unbound, driving a thermal wind. Far-ultraviolet radiation usually dominates, with extreme ultraviolet important near massive stars.

The FRIED grid (Haworth et al. 2018) provides EPE mass loss rates across host star mass, radiation field, disc radius, and mass. Coleman & Haworth (2020) used a simplified scaling due to the high minimum radiation field in FRIED, which yields mass loss rates too large for Peter Pan discs around M dwarfs.

IPE, driven by the host star's radiation, shows mass loss nearly independent of stellar mass but strongly dependent on X-ray luminosity (Owen et al. 2012; Picogna et al. 2019). X-ray luminosity scales with stellar mass (Flaccomio et al. 2012), giving a mass loss scaling exponent $\beta \approx 1.87$. X-ray luminosity decreases by an order of magnitude from 1 to 110 Myr, approximately as $L_X \propto t^{-2/5}$ after radiative core formation (Johnstone et al. 2021). We adopt:

$$L_X(t) = \begin{cases} L_{X,0} & t < 1\mathrm{Myr} \\ L_{X,0}\left(\dfrac{t}{1\mathrm{Myr}}\right)^{-2/5} & t > 1\mathrm{Myr}. \end{cases} \tag{B.4}$$

Here, $L_{X,0}$ was adopted from Flaccomio et al. (2012).

In Figure B.3, we present an example of how the FRIED grid can be used in combination with SeBa and VADER. The thick curves are for a 16 $M_\odot$ star irradiating a 0.01 $M_\odot$ disk around a 1 $M_\odot$ star at a distance of 0.1 pc. The thin curves are calculated for the same star but then at a distance of 1 pc.

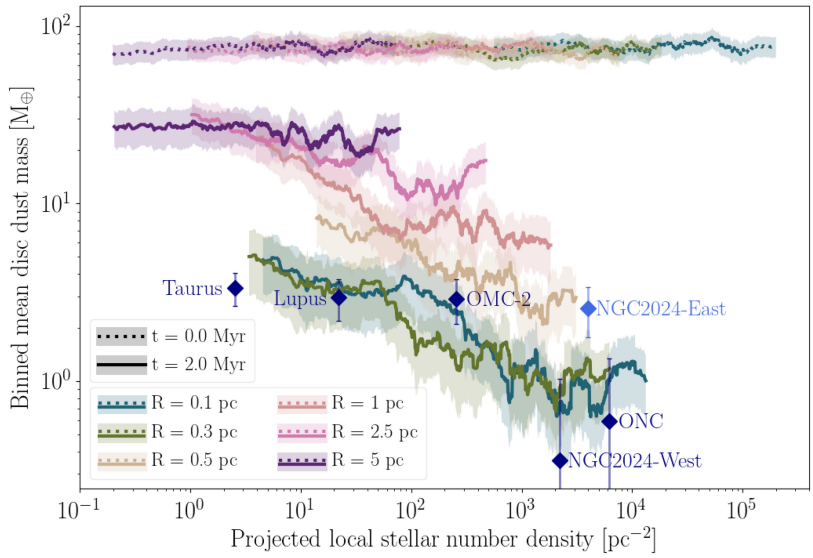


**Figure B.2.** Binned mean disc dust mass versus local stellar density, projected in two dimensions. Mean values use bins of 100 stars. Observational comparisons are included from Lupus, ONC, OMC-2, Taurus, and NGC 2024. Adapted with permission from Concha-Ramírez et al. (2021). © 2020 The Authors. Published by Oxford University Press on behalf of the Royal Astronomical Society

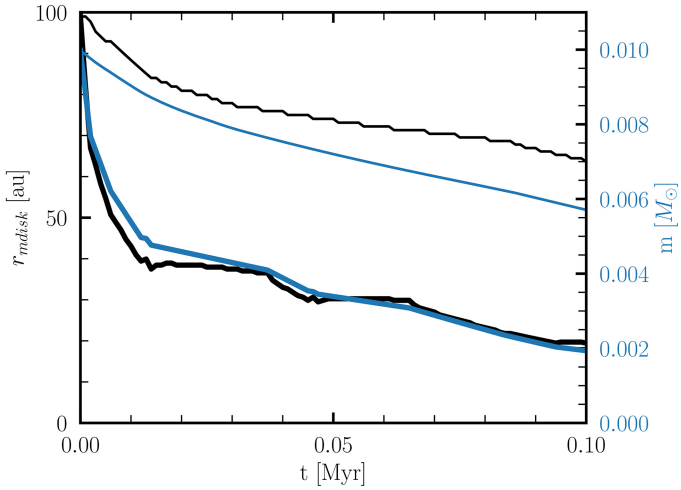


**Figure B.3.** Demonstration of how SeBa, VADER and the Fried grid can lead to interesting results in terms of the EPE of a circumstellar disk due to a nearby massive star. The blue curves adopt the disk mass as a function of time (right-hand axis scale), and the black curve give the disk radius (left). A script to generate this figure can be found at `amuse.examples.ppd_minimal`.

### B.2.5 PACE: Planet Accretion and Evolution

Planet growth in our model proceeds via core accretion (Perri & Cameron 1974), beginning with the formation of a solid core that grows by accreting pebbles. Pebble

accretion (Ormel et al. 2015) is efficient for Moon- to Ceres-sized embryos and is described by

$$\dot{M}_{\rm peb} = \varepsilon 2\pi r u_{\rm peb} \Sigma_{\rm peb}, \tag{B.5}$$

where $\varepsilon$ is the pebble accretion efficiency and $u_{\rm peb}$ the pebble drift velocity, both of which depend on the Stokes number St. These are calculated in a mass-weighted fashion from the two-population dust model:

$$\mathrm{St} = \left(1 - f_{\rm m}\right)\mathrm{St}_0 + f_{\rm m}\,\mathrm{St}_1, \quad u_{\rm peb} = \left(1 - f_{\rm m}\right)v_0 + f_{\rm m}\,v_1. \tag{B.6}$$

Accretion ceases when the planet reaches the pebble isolation mass (Bitsch et al. 2018), at which point it creates a pressure bump that halts pebble inflow:

$$\begin{aligned} M_{\rm iso} = 25 M_\oplus \left(\frac{h}{0.05}\right)^3 \left[0.34\left(\frac{-3}{\log_{10}\alpha_t}\right)^4 + 0.66\right] \\ \times \left[1 - \frac{\partial \ln P/\partial \ln r + 2.5}{6}\right]\frac{M_\star}{M_\odot}. \end{aligned} \tag{B.7}$$

Gas accretion onto the core then follows, initially regulated by Kelvin–Helmholtz contraction (Ikoma et al. 2000):

$$\dot{M}_{\rm KH} = 10^{-5}\left(\frac{M_{\rm p}}{M_\oplus}\right)^4 \left(\frac{\kappa}{0.1\ \mathrm{m^2 kg^{-1}}}\right)^{-1} M_\oplus\ \mathrm{yr}^{-1}, \tag{B.8}$$

but may later be limited by gas inflow across the planetary Hill sphere (Tanigawa & Tanaka 2016):

$$\dot{M}_{\rm Hill} = \frac{0.29}{3\pi h^4}\left(\frac{M_{\rm p}}{M_\star}\right)^{4/3} \frac{\dot{M}_{\rm g}}{\alpha_{\rm acc}} \frac{\Sigma_{\rm gap}}{\Sigma_{\rm g}}, \tag{B.9}$$

with gap depth factor

$$K = \left(\frac{M_{\rm p}}{M_\star}\right)^2 h^{-5}\alpha_{\rm t}^{-1}. \tag{B.10}$$

The final gas accretion rate is taken as the minimum of all limiting rates:

$$\dot{M}_{\rm GA} = \min\left[\dot{M}_{\rm KH},\ \dot{M}_{\rm Hill},\ \dot{M}_{\rm g}\right]. \tag{B.11}$$

Figure B.4 illustrates the diversity observed in planetary systems from two runs using PACE, one without dynamics, and one in which we took the dynamical evolution of the stars and planetary systems into account. In the latter run, we also incorporated the FRIED grid (Section B.2.4) to affect the evolution of circumstellar disks using VADER (Section B.2.3). In Figure B.4, we include the solar system for comparison: the orbital parameters range over more than four orders of magnitude, provoking many questions about their formation, evolution, the role of birth

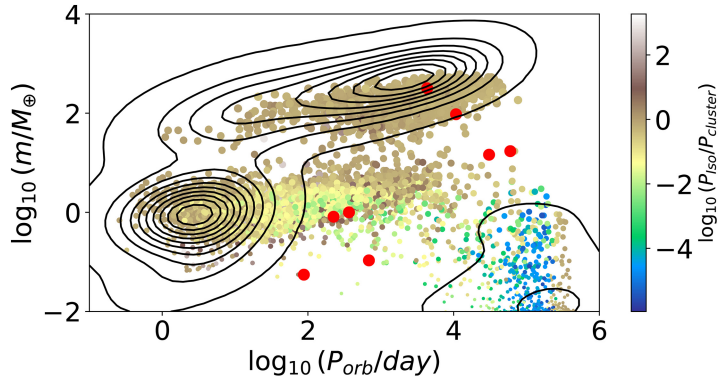


**Figure B.4.** Simulated planet population for isolated stars (contours) and for stars in a cluster (bullets) (Huang et al. 2024). The initial conditions are identical in both cases, but the planetary systems in a clustered environment are subject to EPE onto the planet-forming disk. The colors indicate the logarithmic change in the planetary system's orbital period from isolation to perturbed.

environment and later isolated secular evolution. So far, research has focused on the isolated formation and evolution of planetary systems, but it has become clear that stars form clustered and that this clustering has a long-term effect on the evolution of planetary systems.

### B.2.6 Packages Not Yet Used in Scientific Production

Several packages are being developed as this book goes to print. Some of them have an operational interface, but are not yet tested in production simulations. Experience has taught us that software not used for producing scientific results is buggy, at best. We therefore do not necessarily recommend using these packages, but by the time this book is available some of them may have a working operational interface and will have been tested in practice.

They may also inspire the reader to start another project, and contribute to the framework. The more people contribute, the better and more useful the software becomes.

### B.2.7 Solar System Initial Conditions

As part of a Master's project, Nadia Kempers developed a collection of routines to generate a realization of the solar system as we know it, based on available ephemerides from NASA and JPL. The script generates a current solar system particle set, allowing us to integrate it in a larger environment, or to study the orbital evolution of specific objects. It enables to selection of specific objects, planets, moons, planetesimals, comets, etc. Although, there are a number of hidden assumptions, such as the mass of objects, or their sizes, where observational data is unavailable, it is, so far, the most complete solar system orbital generator we have seen.

### B.2.8 The Interface to Interstellar Nonequilibrium Chemistry

As part of a Master's project, Jelmer Stroo developed an interface to the UCLCHEM. UCLCHEM (Holdship et al. 2017; Keil et al. 2022) is a gas–grain chemical code designed for astrochemical modeling across diverse environments, such as molecular clouds, protostellar cores, and shocks. Unlike many codes that focus solely on gas-phase reactions, UCLCHEM incorporates comprehensive gas–grain processes, including freeze-out, nonthermal and thermal desorption, and grain-surface reactions

UCLCHEM models both gas-phase and grain-surface reactions, treating molecules on grains as distinct species from their gas-phase counterparts. Nonthermal desorption processes include molecule release due to $H_2$ formation, cosmic ray interactions, and UV irradiation. Thermal desorption is uniquely modeled as multiple events, reflecting laboratory findings, rather than a single peak as in other codes. The code propagates species abundances through a user-defined network of reactions, adapting to the physical conditions of the modeled environment.

Written in Fortran 90, UCLCHEM utilizes the DVODE algorithm to solve the resulting system of rate equations. The code is optimized for high-density, low-temperature environments (e.g., protostellar cores and shocks), and is less suitable for hot, low-density conditions where gas–grain interactions are minimal.

When used within AMUSE, it is essential to ensure that simulations remain within UCLCHEM's intended parameter space to maintain accuracy.

Although not yet used in production, we envision the use of UCLCHEM for protostellar core chemistry, shock and jet-driven desorption, chemistry in Seyfert galaxies, prebiotic molecule formation, effects of cosmic rays in dense regions, and machine learning emulators for astrochemical networks.

While this proof-of-concept simulation has notable limitations, it demonstrates the significant potential of integrating UCLCHEM with AMUSE for advanced scientific studies. The current implementation offers several features that can support more sophisticated simulations.

In hydrodynamical simulations, sink particles are employed to represent high-density regions that cannot be resolved at the simulation's spatial resolution (Bate et al. 1995). These are modeled as point masses that interact gravitationally but not hydrodynamically, and can accrete surrounding material. AMUSE facilitates the creation, management, and tracking of sink particles, including the chemical abundances of accreted material, which is valuable for core-scale modeling.

The choice of hydrodynamics code within AMUSE influences the physical processes included in a simulation. For example, the SPH code `Fi` is straightforward to use but lacks magnetic fields, turbulence, and molecular cooling. In contrast, codes such as `Athena2` (AMR: Stone et al. 2008, 2020) and `Phantom` (SPH: Price et al. 2018) support magnetohydrodynamics, with `Phantom` also incorporating a basic chemical network for molecular cooling—an important process in molecular clouds.

AMUSE's multiphysics framework allows for the integration of additional physical processes. This includes *N*-body codes for simulating planetary embryos, coupling chemical codes with stellar evolution models to study stellar winds, or

combining stellar evolution, hydrodynamics, and radiative transfer to investigate radiative feedback in the interstellar medium. While increasing the complexity and computational cost, AMUSE's modular design streamlines the coupling of diverse codes, now including astrochemistry, enabling comprehensive and flexible astrophysical simulations.

In Figure B.5 we show the result of a simulated molecular cloud on the verge of collapsing and making the first stars. The setup for the on-the-fly simulation is similar to that in Priestley et al. (2023a): two spherical molecular clouds, each with mass $M = 10^4\,\mathrm{M}_\odot$, radius of $r = 19$ pc, and a uniform number density of $n_H = 10\,\mathrm{cm}^{-3}$, are initialized on either side of the $x$-axis with their centers at $\pm r$, so that the clouds' edges touch at $x = 0$. The clouds are then each given a bulk motion of $v = \pm 7\,\mathrm{km}^{-1}$ to cause them to collide. The initial gas temperature is set at $T = 300$ K. The SPH code `fi` (Pelupessy et al., 2004) is used to simulate the hydrodynamics of the molecular gas. Each molecular cloud consists of 100,000 particles, for a mass resolution of 0.1 $\mathrm{M}_\odot$ per particle. Time steps of $4.4 \times 10^3$ yr are used between steps of the hydrodynamical and chemical codes, and the simulation is stopped at a time of $t_{end} = 5$ Myr.

As in Priestley et al. (2023a), the UV radiation field is set at 1.7 times the Habing (1968) field, and the cosmic ray ionization rate at $\xi = 10^{-16}\,\mathrm{s}^{-1}$. While the assumption of a constant cosmic ray ionization rate is common in similar works (Priestley et al. 2024, 2023b), recent work has shown that this rate varies drastically based on the environment (see Luo et al. 2023) for diffuse clouds, and for different locations within star formation regions (Sabatini et al. 2023). Though the AMUSE framework is capable of simulating a variable cosmic ray field, in this proof-of-concept simulation the cosmic ray ionization rate is kept constant. This is done to make comparison with earlier work easier, and also to reduce the complexity of this simulation. Due to this same argument of simplicity, turbulence and magnetic fields are not included either.

In the center of the cloud, substructures with higher densities are visible, while near the outside scattered particles that were likely expelled from the clouds during

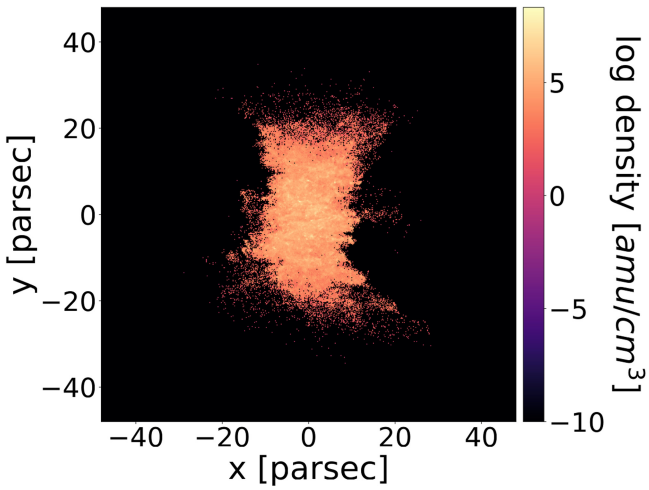


**Figure B.5.** Projected view of a simulated molecular cloud.

the collision can be seen. This distinction is also visible in Figure B.5, which shows the distribution of the gas temperature as a function of the density for the chemical particles at the end of the simulation. As expected, the bulk of the particles are at relatively high density and low temperature, while lower-density particles, near the outside of the cloud, have higher temperatures. The relatively low number of particles prevents a more comprehensive analysis of this distribution. The temperatures of the full set of hydrodynamical particles would allow such an analysis, but (due to an error) they are unfortunately not accessible. Regardless, this distribution is qualitatively in line with expectations.

Unexpectedly, the simulation shows relatively low values for the number density. Priestley et al. (2023a) reached number densities up to $10^6$ cm$^{-3}$ with a very similar setup, which is comparable to observed densities in dense cores. The fact that this simulation does not reproduce these values indicates it is missing some crucial components needed to form these dense structures. Unlike Priestley et al. (2023a), the simulation in this work does not contain turbulence, magnetic fields, or sink particles. Turbulence and magnetic field effects together cause an effect named reconnection diffusion, which is able to mediate cloud collapse. This simulation also does not contain sink particles to model the areas with densities beyond the simulation's resolution. The mass resolution is also higher than in Priestley et al. (2023a): they used a mass resolution of 0.005 $M_\odot$ per particle.

The hydrodynamics of the simulation have a clear effect on the chemistry. The jump in density caused by the collision of the clouds translates to higher abundances for a wide variety of gas-phase molecules. The specific molecules in this figure were chosen to allow for comparison with Priestley et al. (2023b), who present their results for the same selection of molecules, all in the gas phase. It should be noted, however, that the abundance on the grain surface for each of these molecules is also available in this simulation, as well as every other molecule in the network. These two are chosen because they are commonly observed in the gas phase. This shows the successful coupling between the hydrodynamics and chemistry in this simulation. The abundances show a similar spatial distribution to the hydrogen density shown in Figure B.6.

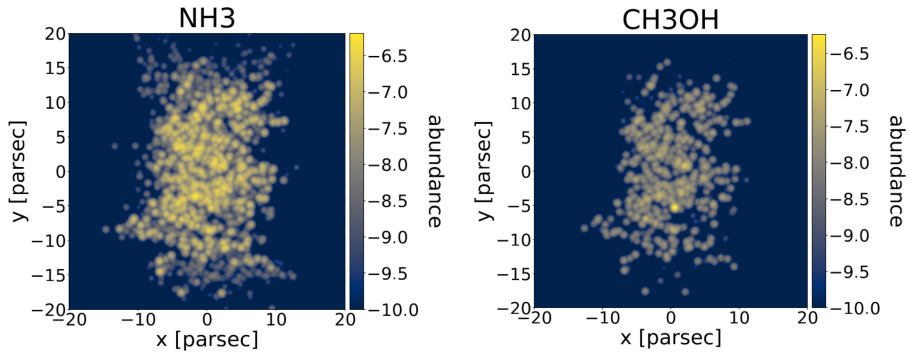


**Figure B.6.** Projected distribution of abundances for $NH_3$ and $CH_3OH$.

### B.2.9 `Tidymess`: Tidal Dynamics of Multibody Extrasolar Systems

As part of a Master's project, Tünde Meyer developed an interface to the `Tidymess` code. `Tidymess` (Tidal Dynamics of Multibody Extrasolar Systems; Boekholt & Correia 2023a, b) is an open-source *N*-body code designed to model the complex dynamical evolution of systems where tidal forces and deformations play a critical role, such as in planetary systems, moons, stars, and compact objects. `Tidymess` enables detailed, efficient, and flexible modeling of tidal dynamics in multibody astrophysical systems, filling a unique niche for studying chaotic and nonlinear tidal evolution.

`Tidymess` implements the Creep deformation law (Norton 1929; Bailey 1935), allowing each body's tidal response to be characterized by its fluid Love number and relaxation time. This approach is more realistic for long-term evolution and can handle both linear and nonlinear tidal regimes. The code models bodies as ellipsoids, accounting for both tidal and centrifugal deformations. The gravitational field is calculated up to quadrupole order, enabling accurate computation of accelerations and torques. Orbits, spins, and deformations are integrated self-consistently using a fourth-order symplectic method. A novel aspect is that deformation integration uses a time step based on orbital timescales, independent of spin or relaxation times, which improves efficiency and consistency across different tidal regimes `Tidymess` is particularly suited for systems with many bodies, chaotic orbital evolution, and strong, nonlinear tidal interactions

## References


Bailey, R. W. 1935, PIME, 131, 1

Bal, H. E., Maassen, J., van Nieuwpoort, R., et al. 2010, Compr, 43, 54

Bate, , et al. 1995, MNRAS, 277, 362

Bitsch, B., Morbidelli, A., Johansen, A., et al. 2018, A&A, 612, A30

Boekholt, T. C. N., & Correia, A. C. M. 2023a, MNRAS, 522, 2885

Boekholt, T. C. N., & Correia, A. C. M. 2023b, TIDYMESS: TIdal DYnamics of Multi-body ExtraSolar Systems, Astrophysics Source Code Library, ascl:2306.004

Coleman, G. A. L., & Haworth, T. J. 2020, MNRAS, 496, L111

Concha-Ramírez, F., Wilhelm, M. J. C., Portegies Zwart, S., van Terwisga, S. E., & Hacar, A. 2021, MNRAS, 501, 1782

Dagum, L., & Menon, R. 1998, ICSEn, 5, 46

Drost, N., Maassen, J., van Meersbergen, M., et al. 2012, IPDPS 2012: 26th {IEEE} International Parallel and Distributed Processing Symposium Workshops (Piscataway, NJ: IEEE) 150

Flaccomio, E., Micela, G., & Sciortino, S. 2012, A&A, 548, A85

Gay, W. W. 2000, Linux Socket Programming: By Example (Indianapolis, IN: Que Corp.)

Gosling, J., Joy, B., & Steele, G. L. 1996, The Java Language Specification (1st ed.; Boston, MA: Addison-Wesley)

Gosling, J., Joy, B., Steele, G. L., Bracha, G., & Buckley, A. 2014, The Java Language Specification, Java SE 8 Edition (1st ed.; Boston, MA: Addison-Wesley)

Gropp, W. 2002, MPICH2: A New Start for MPI Implementations 7 (Berlin: Springer)

Gropp, W., Lusk, E., Doss, N., & Skjellum, A. 1996, ParC, 22, 789

Habing, H. 1968, BAN, 19, 421

Hamers, A. S., & Portegies Zwart, S. F. 2016, MNRAS, 459, 2827
Haworth, T. J., Clarke, C. J., Rahman, W., Winter, A. J., & Facchini, S. 2018, MNRAS, 481, 452
Holdship, J., Viti, S., Jiménez-Serra, I., Makrymallis, A., & Priestley, F. 2017, AJ, 154, 38
Huang, S., Portegies Zwart, S., & Wilhelm, M. J. C. 2024, A&A, 689, A338
Ikoma, M., Nakazawa, K., & Emori, H. 2000, ApJ, 537, 1013
Johnstone, C. P., Bartel, M., & Güdel, M. 2021, A&A, 649, A96
Keil, M., Viti, S., & Holdship, J. 2022, ApJ, 927, 203
Kozai, Y. 1962, AJ, 67, 591
Krumholz, M. R., & Forbes, J. C. 2015, A&C, 11, 1
Lidov, M. L. 1962, Planet. Space Sci., 9, 719
Luo, G., Zhang, Z.-Y., Bisbas, T. G., et al. 2023, ApJ, 946, 91
Lynden-Bell, D., & Pringle, J. E. 1974, MNRAS, 168, 603
Nethercote, N., & Seward, J. 2007, PLDI '07: Proc. 28th SIGPLAN Conference on Programming Language Design and Implementation ed. K. S. McKinley, & J. Ferrante (New York: ACM) 89
Norton, F. H. 1929, The Creep of Steel at High Temperatures (New York: McGraw-Hill)
Ormel, C. W., Shi, J.-M., & Kuiper, R. 2015, MNRAS, 447, 3512
Owen, J. E., Clarke, C. J., & Ercolano, B. 2012, MNRAS, 422, 1880
Peterson, P. 2009, IJCSE, 4, 296
Pelupessy, F. I., van der Werf, P. P., & Icke, V. 2004, A&A, 422, 55
Perri, F., & Cameron, A. G. W. 1974, Icar, 22, 416
Picogna, G., Ercolano, B., Owen, J. E., & Weber, M. L. 2019, MNRAS, 487, 691
Portegies Zwart, S., McMillan, S., Harfst, S., et al. 2009, NewA, 14, 369
Price, D. J., Wurster, J., Tricco, T. S., et al. 2018, PASA, 35, e031
Priestley, F. D., Clark, P. C., Glover, S. C. O., et al. 2023a, MNRAS, 526, 4952
Priestley, F. D., Clark, P. C., Glover, S. C. O., et al. 2023b, MNRAS, 524, 5971
Priestley, F. D., Clark, P. C., Glover, S. C. O., et al. 2024, MNRAS, 531, 4408
Robbins, A. 2005, GDB - Pocket Reference: Debugging Quickly and Painlessly (Sebastopol, CA: O'Reilly & Associates)
Sabatini, G., Bovino, S., & Redaelli, E. 2023, ApJL, 947, L18
Stone, J. M., Gardiner, T. A., Teuben, P., Hawley, J. F., & Simon, J. B. 2008, ApJS, 178, 137
Stone, J. M., Tomida, K., White, C. J., & Felker, K. G. 2020, ApJS, 249, 4
Tanigawa, T., & Tanaka, H. 2016, ApJ, 823, 48
Toonen, S., Nelemans, G., & Portegies Zwart, S. 2012, A&A, 546, A70
Toonen, S., Portegies Zwart, S., Hamers, A. S., & Bandopadhyay, D. 2020, A&A, 640, A16
Toonen, S., Hamers, A., & Portegies Zwart, S. 2016, ComAC, 3, 6
van der Walt, S., Colbert, S. C., & Varoquaux, G. 2011, CISE, 13, 22
Vasiliev, E. 2018, AGAMA: Action-based Galaxy Modeling Framework, Astrophysics Source Code Library, ascl:1805.008
Vasiliev, E. 2019, MNRAS, 482, 1525
von Zeipel, H. 1910, AN, 183, 345

# QUERY FORM

BOOK TITLE: Astrophysical Recipes (Second Edition)

AUTHOR: Simon Portegies Zwart, Steve McMillan and Steven Rieder

CHAPTER TITLE: Back matter

**Page 15**

---

Q1
Please provide citation for Figure B.2.

Appendix B.2 page B-14: add the text indicated
in response.txt file